TechnoChat Application Overview

Generated on 11 April 2026 from the current codebase in `/Users/kaivanshah/Documents/Techno_Chat_2/techno_chat`.

1. Purpose

TechnoChat is an internal Django-based AI application for Technostacks. It allows contributors to:

- sign in with `@technostacks.com` accounts
- complete their profile on first login
- upload supported files
- create chat sessions
- chat with uploaded files using RAG
- ask general AI Assistant questions
- run real-time Web Search prompts
- generate or edit images

It also includes a dedicated admin portal for managing admins, contributors, profiles, files, sessions, and messages.

2. Main User Functionalities

Authentication

- Contributor login at `/`
- Logout at `/logout/`
- First-login redirect to `/profile/` if profile is incomplete
- Admin login at `/admin-login/`
- Admin first-login redirect to `/admin-profile/`

Profile Management

- Contributor profile page at `/profile/`
- Admin profile page at `/admin-profile/`
- Profile forms collect first name, surname, username, position, and team
- Username and email rules are validated before save

File Management

- Files page at `/files/`
- Upload endpoint at `/files/upload/`
- Supported upload families:
- PDF
- DOCX
- PPTX
- XLSX / XLS
- CSV
- TXT
- Markdown
- PNG / JPG / JPEG / WEBP / SVG
- File status lifecycle:
- `pending`
- `completed`
- `failed`

Chat and Session Management

- Session list redirect at `/chat/`
- Session creation page at `/chat/new/`
- Session detail page at `/chat/<session_id>/`
- Chat send endpoint at `/chat/<session_id>/send/`
- Source viewer endpoint at `/chat/page-render/`

Session types:

- `chat_with_file`
- `general_chat`

Chat modes:

- `rag`
- `ai_assistant`
- `web_search`
- `image_generation`

RAG / Document Chat

- Retrieves file-linked chunks from Pinecone
- Builds context-aware answers with source locations
- Supports page, slide, sheet, row, line, and markdown section references
- Handles greetings, follow-up questions, exact wording requests, summary requests, list requests, and numeric filter requests

AI Assistant

- Answers without document context
- Uses conversation state and recent chat history where needed
- Returns plain answers without document sources

Web Search

- Uses Serper search results
- Synthesizes a final answer through the selected LLM
- Returns clickable web links as sources

Image Generation

• Text-to-image generation
• Image-to-image editing
• Supports uploaded image previews and generated image output in chat

3. Admin Portal Functionalities

Admin routes:

• `/admin-login/`
• `/admin-register/`
• `/admin-profile/`
• `/admin-dashboard/`
• `/admin-new-contributor/`
• `/admin-logout/`

Admin dashboard sections:

• Admins
• Admin Profiles
• Contributors
• Profiles
• Files
• Sessions
• Messages

Admin actions covered by the code:

• login and logout
• create new contributor
• edit existing records
• single delete
• bulk delete
• view history panel for edited records

• update file status

• update session-file many-to-many links

4. Frontend Structure

Main user templates:

• `backoffice_engine/templates/login.html`

• `backoffice_engine/templates/base.html`

• `backoffice_engine/templates/home.html`

• `backoffice_engine/templates/profile.html`

• `backoffice_engine/templates/upload.html`

• `backoffice_engine/templates/create_session.html`

• `backoffice_engine/templates/chat.html`

• `backoffice_engine/templates/about_us.html`

Admin templates:

• `backoffice_engine/templates/admin/admin_base.html`

• `backoffice_engine/templates/admin/admin_login.html`

• `backoffice_engine/templates/admin/admin_register.html`

• `backoffice_engine/templates/admin/admin_profile_complete.html`

• `backoffice_engine/templates/admin/admin_dashboard.html`

Key frontend JavaScript files:

• `backoffice_engine/static/base.js`

• `backoffice_engine/static/upload.js`

• `backoffice_engine/static/profile.js`

• `backoffice_engine/static/create_session.js`

• `backoffice_engine/static/chat.js`

• `backoffice_engine/static/admin/admin_dashboard.js`

• `backoffice_engine/static/about_us.js`

Important UI behaviors present in the code:

- theme toggle with `dark`, `light`, and `system`
- profile dropdown open/close
- password eye toggle on login and admin forms
- upload zone show/hide
- chat textarea auto-expand
- disabled send button when empty
- image preview before send
- source viewer modal for RAG sources
- select all / deselect all file selection on create-session page

5. Backend Structure

Core request/response modules:

- `backoffice_engine/views.py`
- `backoffice_engine/admin_views.py`
- `techno_chat/urls.py`

Validation and helpers:

- `backoffice_engine/validators.py`
- `backoffice_engine/helpers.py`
- `backoffice_engine/exceptions.py`
- `backoffice_engine/forms.py`

AI and retrieval services:

- `backoffice_engine/chat_service.py`
- `backoffice_engine/ai_assistant_service.py`
- `backoffice_engine/web_search_service.py`

- `backoffice_engine/image_generation_service.py`
- `backoffice_engine/retrieval_service.py`
- `backoffice_engine/structured_file_service.py`
- `backoffice_engine/query_service.py`
- `backoffice_engine/response_parsing_service.py`

Document ingestion and rendering:

- `backoffice_engine/document_reader.py`
- `backoffice_engine/ingestion_service.py`
- `backoffice_engine/page_render_service.py`
- `backoffice_engine/image_processing_service.py`

External client wrappers:

- `backoffice_engine/clients.py`

Backend integrations visible in the code:

- Django
- PostgreSQL configuration in app settings
- Pinecone for vector storage and reranking
- LangChain text splitting and model wrappers
- Google Gemini models
- Groq-hosted models
- Serper for web search
- OpenAI/Kie image models

Configured model labels visible in the code:

- Gemini 2.5 Pro
- Gemini 2.5 Flash
- Gemini 2.5 Flash-Lite
- Llama 3.3 70B

• Llama 3.1 8B
• GPT OSS 120B

6. Data Model / Tables

The current `backoffice_engine/models.py` file defines 7 core application models:

1. `AdminUser`
2. `User`
3. `AdminProfile`
4. `UserProfile`
5. `File`
6. `ChatSession`
7. `ChatMessage`

Practical table-level responsibilities:

`AdminUser`

• admin authentication identity
• email-based login
• Django `AbstractUser`-based model
• tracks `profile_completed`

`User`

• contributor authentication identity
• email and password storage
• tracks `profile_completed`

`AdminProfile`

• one-to-one with `AdminUser`

• stores display name, username, position, team, and completion status

`UserProfile`

• one-to-one with `User`
• stores display name, username, position, team, and completion status

`File`

• belongs to a contributor
• stores uploaded file path, file type, original filename, and embedding status

`ChatSession`

• belongs to a contributor
• many-to-many with files
• stores title and session type

`ChatMessage`

• belongs to a session
• stores question, answer, selected model, `sources` JSON, and `chat_mode`

Additional framework-managed tables are also relevant at runtime, especially for:

• Django admin logging
• session storage
• auth permissions / groups

7. File Processing Pipeline

The upload pipeline is split into two major layers.

Extraction

`document_reader.py` extracts structured text from supported file types and returns location-aware segments.

Examples:

- PDF: page-based segments, embedded-image OCR, table extraction
- DOCX: paragraph extraction, table extraction, embedded-image OCR
- PPTX: slide extraction, table extraction, image OCR
- XLSX / CSV: row batches plus statistical summary segments
- TXT: line-range segments
- Markdown: heading-based sections
- Image files: VLM description plus visible text extraction
- SVG: SVG text plus visual summary

Ingestion

`ingestion_service.py`:

1. extracts structured segments
2. splits segments into chunks
3. generates dense and sparse embeddings
4. upserts vectors into Pinecone
5. updates file status to completed or failed

8. Source Rendering

The app supports source viewing from chat responses.

• PDF pages can be rendered as page images
• PPTX sources can be rendered through the page render service
• text-based sources can return textual excerpts
• source location metadata can include:
• page index
• slide index
• sheet name
• row start / end
• line start / end
• markdown section name

9. Automated Test Coverage

The recreated suite currently contains 41 passing tests in:

• `backoffice_engine/tests.py`

Coverage groups included:

• Authentication
• File upload and document extraction
• RAG chat
• AI Assistant
• Web Search
• Image Generation
• Session and chat management
• Admin portal workflows
• UI and navigation regression checks

The test command used successfully was:

`python3 manage.py test backoffice_engine --settings=techno_chat.test_settings`

The test settings isolate the run from the main environment by using:

• in-memory SQLite

• a separate test media directory

• disabled app migrations for the test run

10. Current Status

What is confirmed

• The 41 automated tests currently pass in this workspace.

• The added suite gives good regression coverage for the major flows listed above.

What is not guaranteed by a green test run

• A passing suite does not prove every single screen and edge case is perfect.

• Manual browser verification is still recommended for final delivery.

Known code note

The current admin dashboard template contains invalid Django template comparisons at:

• `backoffice_engine/templates/admin/admin_dashboard.html:182`

• `backoffice_engine/templates/admin/admin_dashboard.html:186`

Those lines currently use:

• `{% if edit_record.team==val %}`

In Django templates that should be written with spaces around `==`.

Because you asked me not to change working application code in this test task, I left the app logic untouched and kept the test suite resilient around that issue.

11. How to Run the Tests

From the project root:

`/Users/kaivanshah/Documents/Techno_Chat_2/techno_chat`

run:

python3 manage.py test backoffice_engine --settings=techno_chat.test_settings

Useful optional commands:

Run only one class:

python3 manage.py test backoffice_engine.tests.AuthenticationTests --settings=techno_chat.test_settings

Run only admin tests:

python3 manage.py test backoffice_engine.tests.AdminPortalTests --settings=techno_chat.test_settings

Run Django project checks:

python3 manage.py check

12. Short Operational Summary

TechnoChat is a contributor-facing AI workspace plus an internal admin portal. Contributors can upload documents, create file-based or general chat sessions, use four AI modes, and inspect sources. Admins can manage identities, profiles, files, sessions, and messages from a central dashboard. The backend is organized around Django views plus dedicated AI service modules, while the frontend uses server-rendered templates with page-specific JavaScript and CSS. The current automated suite passes, but there is still at least one known template-level issue that should be cleaned up before calling the whole codebase fully clear.